

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0515

Slot B

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 10%

QUESTIONS:4

Total Marks 20

Duration :60 Minutes Annexure A

Mock Interview

Code of Conduct:

I K.Mounith(192425090) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Mounith

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problemsolving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides wellstructured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	

Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	
-----------------------	----	--	---	------------------------------------	---------------------------------	--

OpenCV Basics & Pipeline Thinking

- Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.

A. OpenCV Pipeline to Read, Process, and Display Multiple Images from a Folder

Objective

- Read multiple images from a folder, process them efficiently, and display the results.
- Pipeline
- Folder → Get Image Files → Read Image → Resize → Process → Display → Next Image
- Step-by-step
- Get image files
- Use Python os or glob to find .jpg, .png, etc.
- Read images
- Use cv2.imread().
- Check image
- If imread() returns None, skip the file.
- Process image
- Resize using cv2.resize().
- Convert to grayscale using cv2.cvtColor().
- Apply required processing such as blur or edge detection.
- Display
- Use cv2.imshow().
- Continue
- Press n for the next image or q to quit.
- Simple Code
- import cv2
- import glob
- files = glob.glob("images/*.*)"
- for file in files:
- img = cv2.imread(file)
- if img is None:
- continue
- img = cv2.resize(img, (640, 480))
- gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
- cv2.imshow("Original", img)

- `cv2.imshow("Processed", gray)`
 - `key = cv2.waitKey(0)`
 - `if key == ord('q'):`
 - `break`
 - `cv2.destroyAllWindows()`
 - Justification and Optimization
 - Resize → reduces memory and processing time.
 - Grayscale → reduces computation.
 - Skip invalid files → prevents program failure.
 - Process one image at a time → avoids loading all images into memory.
 - Challenges
 - Corrupted images
 - Different image sizes
 - Unsupported formats
 - Very large folders
 - High memory usage
2. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.
- A.
- Video File Not Loading Properly in OpenCV
 - Possible Causes
 - Wrong file path
 - Incorrect file name or extension
 - Video codec not supported
 - Corrupted video
 - OpenCV installation problem
 - Incorrect use of `VideoCapture()`
 - Permission/access problems
 - Structured Debugging Approach
 - Step 1: Check the file path
 - `import os`
 - `print(os.path.exists("video.mp4"))`
 - If it returns False, the path is incorrect.
 - Step 2: Open the video
 - `cap = cv2.VideoCapture("video.mp4")`
 - Step 3: Check whether it opened
 - `if not cap.isOpened():`
 - `print("Video cannot be opened")`
 - Step 4: Read a frame
 - `ret, frame = cap.read()`

- if not ret:
 - print("Frame cannot be read")
 - Step 5: Check video properties
 - print(cap.get(cv2.CAP_PROP_FPS))
 - print(cap.get(cv2.CAP_PROP_FRAME_COUNT))
 - Step 6: Try another video
 - If another video works, the original video may have a codec or corruption problem.
 - Step 7: Check installation
 - pip install --upgrade opencv-python
 - Conclusion
 - Debugging should proceed from file path → OpenCV opening → frame reading → video properties → codec/video integrity. This helps identify the exact source of the problem instead of changing code randomly.
3. You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.
- A.
- Overlay Text and Shapes on Live Video
 - Objective
 - Capture live video and add text, rectangles, circles, and lines efficiently.
 - Pipeline
 - Camera → Capture Frame → Detect/Select Region → Draw Shapes → Add Text → Display
 - Important OpenCV Functions

• Function	• Purpose
• cv2.VideoCapture()	• Capture video
• cv2.rectangle()	• Draw rectangle
• cv2.circle()	• Draw circle
• cv2.line()	• Draw line
• cv2.putText()	• Add text
• cv2.imshow()	• Display video
• cv2.waitKey()	• Control loop
 - Simple Code
 - import cv2
 - cap = cv2.VideoCapture(0)
 - while True:
 - ret, frame = cap.read()
 - if not ret:
 - break
 - # Draw rectangle

- `cv2.rectangle(frame, (100, 100), (400, 300),`
 - `(0, 255, 0), 2)`
 - `# Draw circle`
 - `cv2.circle(frame, (250, 200), 50,`
 - `(255, 0, 0), 2)`
 - `# Draw line`
 - `cv2.line(frame, (0, 400), (640, 400),`
 - `(0, 0, 255), 2)`
 - `# Add text`
 - `cv2.putText(frame, "Live Video",`
 - `(20, 40),`
 - `cv2.FONT_HERSHEY_SIMPLEX,`
 - `1, (255, 255, 255), 2)`
 - `cv2.imshow("Video", frame)`
 - `if cv2.waitKey(1) & 0xFF == ord('q'):`
 - `break`
 - `cap.release()`
 - `cv2.destroyAllWindows()`
 - Efficiency
 - Keep the video resolution reasonable.
 - Avoid unnecessary image conversions.
 - Draw directly on the current frame.
 - Use `waitKey(1)` for continuous display.
 - Process only the required region if detection is involved.
 - Conclusion
 - OpenCV drawing functions are lightweight, so text and basic shapes can be added to a live feed with very little processing overhead.
4. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.
- A.
- Image Appears Too Dark When Read
 - Problem
 - An image may appear dark because of:
 - Low exposure during image capture
 - Poor lighting
 - Low pixel intensity
 - Incorrect image processing
 - Improper display or color handling
 - Solution Pipeline

- Dark Image → Grayscale/Color Analysis → Brightness Enhancement → Contrast Enhancement → Display
- Method 1: Brightness Adjustment
- Increase pixel intensity using `cv2.convertScaleAbs()`.
- `bright = cv2.convertScaleAbs(`
- `img,`
- `alpha=1.2,`
- `beta=40`
- `)`
- `alpha` → contrast
- `beta` → brightness
- Method 2: Histogram Equalization
- For a grayscale image:
- `gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`
- `enhanced = cv2.equalizeHist(gray)`
- This improves the contrast between dark and bright regions.
- Method 3: CLAHE
- For uneven lighting, CLAHE is often better:
- `gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`
- `clahe = cv2.createCLAHE(`
- `clipLimit=2.0,`
- `tileGridSize=(8, 8)`
- `)`
- `enhanced = clahe.apply(gray)`
- Recommended Pipeline
- Original Dark Image
- ↓
- Check Image / Color Format
- ↓
- Brightness Adjustment
- ↓
- CLAHE
- ↓
- Enhanced Image
- ↓
- Display / Further Processing
- Why CLAHE?
- Normal histogram equalization can over-enhance some areas. CLAHE works locally, so it improves visibility in dark regions while limiting excessive contrast enhancement.
- Conclusion
- For a uniformly dark image, brightness and contrast adjustment can be used. For an image with uneven lighting, CLAHE is a better choice because it improves local contrast and makes details more visible without excessively brightening the entire image.